

Guia para Completar el Informe Final

Objetivo de esta guia

Esta guia sirve para convertir el borrador actual en un informe final alineado con el enunciado del proyecto del equipo 2:

Software que identifique objetos basicos en imagenes mediante redes neuronales

Estructura recomendada del informe

1. Portada

Mantener:

- universidad;
- escuela;
- nombre del curso;
- nombre del proyecto;
- nombres de los integrantes;
- profesor;
- fecha;
- lugar.

2. Introduccion

Explicar en 2 o 3 parrafos:

- el problema a resolver;
- por que la clasificacion de imagenes es relevante;
- que solucion construyeron;
- que clases maneja el sistema: **cats**, **dogs**, **panda**.

3. Objetivos

Objetivo general

Desarrollar un software capaz de procesar imagenes y clasificarlas en tres tipos de objetos utilizando una red neuronal convolucional basica.

Objetivos especificos

- Preparar y validar el dataset de imagenes.
- Implementar el pipeline de preprocesamiento.
- Entrenar una CNN basica para clasificacion multiclase.
- Clasificar imagenes en **cats**, **dogs** y **panda**.
- Evaluar el modelo con metricas automaticas.
- Documentar limitaciones tecnicas del sistema.

4. Descripcion del dataset

Incluir:

- origen del dataset;
- estructura de carpetas por clase;
- clases utilizadas;
- cantidad total de imagenes;
- distribucion `train`, `val`, `test`;
- limpieza o deduplicacion aplicada.

5. Metodologia de desarrollo

Puedes conservar la idea por fases, pero redactada con mas forma academica:

- Fase 1: validacion del dataset y entorno.
- Fase 2: preprocesamiento y division de datos.
- Fase 3: construccion y entrenamiento de la CNN.
- Fase 4: evaluacion con metricas.
- Fase 5: analisis de limitaciones.

6. Desarrollo por requerimientos

Esta es la parte mas importante. Conviene dividirla exactamente como el proyecto:

6.1 Requerimiento A. Disene un software que procese imagenes

Debes explicar:

- carga del dataset;
- lectura de imagenes;
- normalizacion;
- resize;
- manejo de errores;
- confirmacion de registros cargados.

Evidencia sugerida:

- referencia a `core/phase1.py`;
- referencia a `core/phase2.py`;
- referencia al `manifest`;
- captura o tabla con conteos.

6.2 Requerimiento B. Implemente una red neuronal basica

Debes explicar:

- arquitectura CNN utilizada;
- capas principales;
- hiperparametros;
- entrenamiento por epocas;

- guardado de checkpoints.

Evidencia sugerida:

- referencia a `core/model.py`;
- referencia a `core/phase3_train.py`;
- resultados en `phase3_artifacts`.

6.3 Requerimiento C. Clasifique 3 tipos de objetos

Esta seccion debe quedar muy clara porque corresponde al avance 3.

Debes incluir:

- definicion de las 3 clases del sistema;
- explicacion del proceso de inferencia;
- evidencia de que el modelo clasifica `cats`, `dogs` y `panda`;
- salida del sistema o capturas del dashboard.

Texto base sugerido:

El sistema fue configurado para clasificar imagenes en tres categorias cerradas: `cats`, `dogs` y `panda`. Estas clases se encuentran definidas dentro de la configuracion del proyecto y son utilizadas tanto en el entrenamiento como en la inferencia. A partir de una imagen de entrada, el modelo procesa la imagen, la redimensiona al tamaño configurado, la transforma a tensor y genera una prediccion con probabilidades por clase. Finalmente, el sistema devuelve la clase con mayor probabilidad como resultado de clasificacion.

Evidencia sugerida:

- `core/constants.py`
- `core/phase3_infer.py`
- `app.py`
- capturas de pruebas con una imagen por clase

6.4 Requerimiento D. Genere 3 metricas de precision

Aunque en el cronograma este despues, en el informe final debe aparecer.

Incluir:

- `accuracy`
- `macro_precision`
- `macro_f1`

Tambien puedes sumar:

- `macro_recall`
- `weighted_precision`
- `weighted_recall`
- `weighted_f1`

Debes presentar:

- tabla de metricas;
- interpretacion corta;
- reporte por clase;
- matriz de confusion.

Valores actuales observados en `metrics_test.json`:

- `accuracy`: 0.62
- `macro_precision`: 0.6233
- `macro_recall`: 0.62
- `macro_f1`: 0.6154

6.5 Requerimiento E. Documento 5 limitaciones tecnicas

Desarrolla una subseccion por cada limitacion:

1. Generalizacion limitada a tres clases.
2. Arquitectura CNN basica.
3. Posible sesgo residual del dataset.
4. Falta de calibracion de confianza.
5. Variabilidad segun backend de hardware.

7. Resultados

En esta seccion resume:

- si el modelo entrena correctamente;
- si clasifica tres tipos de objetos;
- principales metricas;
- clase con mejor desempeno;
- clase con menor desempeno.

8. Conclusiones

Redactar de 3 a 5 conclusiones. Ejemplo:

- Se logro implementar un sistema funcional de clasificacion de imagenes para tres clases.
- El pipeline permite reproducibilidad mediante manifest, checkpoints y artefactos de evaluacion.
- La clase `panda` obtuvo mejor desempeno relativo que `cats` y `dogs`.
- El modelo es funcional, pero su arquitectura simple limita la capacidad de generalizacion.

9. Recomendaciones

- usar mas epocas;
- aplicar data augmentation mas robusto;
- probar arquitecturas mas profundas;
- calibrar probabilidades;
- agregar deteccion de desconocidos.

10. Referencias o bibliografia

Agregar fuentes del dataset, librerias y documentacion utilizada.

11. Anexos

Agregar si es posible:

- capturas de interfaz;
- salidas de consola;
- tablas de metricas;
- matriz de confusion;
- rutas de artefactos generados.

Checklist antes de entregar

- El informe incluye secciones **A, B, C, D y E**.
- El Requerimiento C tiene evidencia explicita de las 3 clases.
- El Requerimiento D muestra al menos 3 metricas.
- Se incluyen las 5 limitaciones tecnicas.
- Hay conclusiones y recomendaciones.
- Se citan archivos o evidencias del proyecto.
- La redaccion usa lenguaje academico y consistente.